

JS

REPASO COMPLETO JAVASCRIPT

Para llevar al examen junto a tu código

1 · ANÁLISIS DE CÓDIGO — LOS 3 FRAGMENTOS DEL EXAMEN

Cada fragmento vale **1 punto**. Para el punto completo: (1) nombrar el error, (2) explicar el **concepto técnico**, (3) código corregido.

Fragmento 1 — HTMLCollection no tiene .style

X Error: `getElementsByClassName` devuelve una `HTMLCollection` (lista), no un elemento. Las listas no tienen `.style`.

```
// ERROR
const items = document.getElementsByClassName('item');
items.style.backgroundColor = 'blue';

// CORRECCIÓN A — querySelectorAll + forEach
document.querySelectorAll('.item').forEach(function(item) {
  item.style.backgroundColor = 'blue';
});

// CORRECCIÓN B — bucle for clásico
const items = document.getElementsByClassName('item');
for (let i = 0; i < items.length; i++) {
  items[i].style.backgroundColor = 'blue';
}
```

Concepto: `getElementsByClassName` → `HTMLCollection` (sin `forEach`). `querySelectorAll` → `NodeList` (con `forEach`).

Fragmento 2 — Valor capturado fuera del evento

X Error: `.value` se lee al cargar la página (siempre ""). Hay que leerlo DENTRO del callback, en el momento del evento.

```
// ERROR
const username = document.querySelector('#username').value; // siempre ""

form.addEventListener('submit', (e) => {
  if (username === "") { e.preventDefault(); } // siempre se cumple
});

// CORRECCIÓN — leer .value DENTRO del listener
form.addEventListener('submit', function(e) {
  const username = document.querySelector('#username').value; // valor real
  if (username === "") {
    e.preventDefault();
    alert("El campo es obligatorio");
  }
});
```

Concepto: Scope temporal / closure. El valor de un input hay que leerlo siempre dentro del callback del evento.

Fragmento 3 — Arrow function y this

X Error: Las arrow functions NO tienen su propio `this`. `this` es `undefined` o el objeto global, no el botón.

```
// ERROR
btn.addEventListener('click', () => {
  this.remove(); // TypeError: this no es el botón
});

// CORRECCIÓN A — función regular (this = elemento que dispara el evento)
btn.addEventListener('click', function() {
  this.remove(); // this = btn
});
```

```
});  
  
// CORRECCIÓN B – referenciar la variable directamente  
btn.addEventListener('click', () => {  
    btn.remove(); // válido y claro  
});
```

Concepto: Arrow functions heredan this del scope léxico. Funciones regulares tienen su propio this que apunta al elemento del evento.

2 · SELECCIÓN DE ELEMENTOS DEL DOM

Método	Devuelve	forEach	Cuándo usarlo
getElementById('id')	Un elemento / null	—	ID único — el más rápido
querySelector('.clase')	Primer elemento	—	Selector CSS flexible
querySelectorAll('li')	NodeList (estático)	✓ Sí	Todos los que coincidan
getElementsByClassName	HTMLCollection (vivo)	✗ No	Evitar — usar querySelectorAll

```
// Referencia a elementos al inicio del archivo (patrón correcto)
const input      = document.getElementById('guest-name');
const addBtn     = document.getElementById('add-btn');
const errorMsg   = document.getElementById('error-msg');
const guestList  = document.getElementById('guest-list');

// Buscar dentro de un elemento
const countSpan = option.querySelector('.vote-count');

// Convertir NodeList a Array (para .sort())
const items = [...document.querySelectorAll('#word-list li')];
```

■ **NUNCA** uses `getElementById` fuera de las declaraciones iniciales. Llama siempre a las variables declaradas arriba.

3 · EVENTOS Y `addEventListener`

Evento	Cuándo se dispara	Uso típico
'click'	Al hacer clic	Botones, eliminar
'input'	Cada letra que escribe el usuario	Búsqueda en tiempo real
'change'	Al cambiar y perder foco / checkbox	Checkboxes de tareas
'submit'	Al enviar un	Formularios — siempre con <code>e.preventDefault()</code>

```
// Patrón estándar – botón
addBtn.addEventListener('click', function() {
  // lógica aquí
});

// Formulario – SIEMPRE e.preventDefault() primera línea
form.addEventListener('submit', function(e) {
  e.preventDefault(); // evita recargar la página
  // validar y procesar
});

// Búsqueda en tiempo real
searchInput.addEventListener('input', function() {
  const query = searchInput.value.toLowerCase();
  // filtrar elementos
});
```

4 · VALIDACIÓN — PATRÓN CON return TEMPRANO

La validación siempre sigue el mismo orden: comprobar → mostrar error → **return** → continuar si todo ok.

```
addBtn.addEventListener('click', function() {
  const valor = input.value.trim(); // .trim() elimina espacios

  // 1. Campo vacío
  if (valor === '') {
    errorMsg.textContent = 'El campo no puede estar vacío.';
    return; // ← CORTA la función aquí, no sigue
  }
  // 2. Longitud mínima
  if (valor.length < 3) {
    errorMsg.textContent = 'Mínimo 3 caracteres.';
    return;
  }
  // 3. Longitud exacta (teléfono)
  if (valor.length !== 9) {
    errorMsg.textContent = 'El teléfono debe tener 9 dígitos.';
    return;
  }
  // 4. Número válido y positivo
  const precio = parseFloat(priceInput.value);
  if (isNaN(precio) || precio <= 0) {
    errorMsg.textContent = 'El precio debe ser mayor que 0.';
    return;
  }
  // 5. Rango numérico (notas 0-10)
  if (nota < 0 || nota > 10) {
    errorMsg.textContent = 'La nota debe estar entre 0 y 10.';
    return;
  }

  // Si llegamos aquí: todo correcto
  errorMsg.textContent = ''; // limpiar error anterior
  // ... hacer la acción
});
```

Tabla de validaciones más comunes

Qué validar	Código
Campo vacío	valor === '' o valor.trim() === ''
Longitud mínima	valor.length < 3
Longitud exacta	valor.length !== 9
Número válido	isNaN(num) num <= 0
Rango numérico	num < 0 num > 10
Email básico	!email.includes('@') !email.includes('.')
Duplicado en lista	li.dataset.word === valor.toLowerCase()

Calificaciones por nota (Registro de alumnos)

```
let notaCalificativa = ''; // let, no const - se reasigna
if (nota >= 0 && nota <= 4.9) { notaCalificativa = 'Suspenso'; }
if (nota >= 5 && nota <= 6.9) { notaCalificativa = 'Aprobado'; }
if (nota >= 7 && nota <= 8.9) { notaCalificativa = 'Notable'; }
if (nota >= 9 && nota <= 10) { notaCalificativa = 'Sobresaliente'; }
// && (AND) en los tramos - || (OR) siempre es TRUE
```

5 · CREAR Y ELIMINAR NODOS — PATRÓN COMPLETO

■ Orden obligatorio: `createElement` → configurar → `appendChild` hijos → `appendChild` a la lista.

```
// ■■■ PATRÓN COMPLETO (lo que el profesor pide) ■■■■■■■■■■■■  
const li = document.createElement('li');  
  
// Guardar datos en dataset – para recuperarlos sin parsear el texto  
li.dataset.price = precio;           // li.dataset.price al leerlo  
li.dataset.grade = notaAlumno.value; // parseFloat(li.dataset.grade)  
li.dataset.nombreContacto = nombre;   // li.dataset.nombreContacto  
  
// Texto visible  
const textoSpan = document.createElement('span');  
textoSpan.textContent = nombre + ' - ' + precio.toFixed(2) + '€';  
li.appendChild(textoSpan);            // span DENTRO del li  
  
// Botón eliminar con CLOSURE – recuerda su propio li  
const deleteBtn = document.createElement('button');  
deleteBtn.textContent = 'Eliminar';  
deleteBtn.addEventListener('click', function() {  
    li.remove();             // elimina ESTE li (closure)  
    actualizarTotal();        // recalcular después de eliminar  
});  
li.appendChild(deleteBtn); // botón DENTRO del li  
  
// Añadir el li a la lista UL  
listaUL.appendChild(li);  
  
// Limpiar campos  
input.value = '';  
input.focus();  
  
// Llamar función de actualización  
actualizarTotal();
```

✓ CLOSURE: cada deleteBtn recuerda su propio li porque fue creado en su scope. Por eso elimina el correcto.

Funciones de actualización — patrón estándar

[illegible]

6 · MOSTRAR, OCULTAR Y FILTRAR ELEMENTOS

[illegible]

7 · CLASES CSS DESDE JAVASCRIPT

```
li.classList.add('completada');           // añadir clase
li.classList.remove('completada');         // quitar clase
li.classList.contains('completada');       // true / false


// ■■ PATRÓN: botón activo ██████████
function marcarActivo(btn) {
    filterAll.classList.remove('activo');
    filterPend.classList.remove('activo');
    filterDone.classList.remove('activo');
    btn.classList.add('activo');
}


// ■■ PATRÓN: checkbox (Lista de tareas) ██████████
checkbox.addEventListener('change', function() {
    if (checkbox.checked) {
        li.classList.add('completada');
    } else {
        li.classList.remove('completada');
    }
    actualizarContador();
    aplicarFiltro();
});
```

8 · NÚMEROS, STRINGS Y MATH

Conversión de tipos

```
parseFloat('12.5') // → 12.5 (decimal)
parseInt('12')     // → 12 (entero)
isNaN('hola')      // → true (no es número)
isNaN(12.5)        // → false (sí es número)
isNaN('')          // → false ■ string vacío = 0

(12.333).toFixed(2) // → '12.33' (string con 2 decimales)
parseFloat((12.333).toFixed(2)) // → 12.33 si necesitas número
```

Strings — métodos esenciales

```
' hola '.trim()      // → 'hola' (quita espacios)
'HOLA'.toLowerCase() // → 'hola'
'manzana'.includes('man') // → true
'manzana'.length     // → 7
'manzana'.substring(0, 4) // → 'manz' (primeros 4 chars)
```

Math — números aleatorios

```
Math.random()          // decimal 0 a 0.999...
Math.floor(2.9)         // → 2 (redondear abajo)
Math.round(2.5)         // → 3 (redondear normal)

// Entero aleatorio entre 0 y N-1:
Math.floor(Math.random() * N)

// Ejemplos reales:
Math.floor(Math.random() * 100) + 1 // 1 a 100 (adivina el número)
Math.floor(Math.random() * 3)       // 0, 1 o 2 (quiniela)
Math.floor(Math.random() * 5)       // 0 a 4
```

Arrays

```
const resultados = ['1', 'X', '2'];
const res = resultados[Math.floor(Math.random() * 3)]; // elemento aleatorio

// forEach – recorrer
items.forEach(function(li, i) { // i = índice (0,1,2...)
  total += parseFloat(li.dataset.price);
});

// Convertir NodeList a Array para .sort()
const arr = [...document.querySelectorAll('li')];
arr.sort(function(a, b) {
  if (a.dataset.word < b.dataset.word) return -1;
  if (a.dataset.word > b.dataset.word) return 1;
  return 0;
});
arr.forEach(function(li) { wordList.appendChild(li); }); // reordena en DOM
```

9 · VARIABLES DE ESTADO Y CLOSURES

Variables fuera del listener — persisten entre pulsaciones

```
// ✓ FUERA del listener — persiste
let totalQuinielas = 0;
let filtroActivo    = 'todas';
let juegoActivo     = true;

btnGenerar.addEventListener('click', function() {
  totalQuinielas++;           // se acumula en cada clic
  historialSpan.textContent = totalQuinielas;
});

// ✗ DENTRO del listener — se reinicia en cada clic
btn.addEventListener('click', function() {
  let count = 0; // siempre empieza en 0 — ERROR clásico
  count++;      // siempre muestra 1
});
```

Closure — por qué funciona el botón Eliminar

```
addBtn.addEventListener('click', function() {
  const li = document.createElement('li'); // cada clic crea UN li diferente

  const deleteBtn = document.createElement('button');
  deleteBtn.addEventListener('click', function() {
    li.remove(); // esta función RECUERDA su propio li
                // aunque el listener de addBtn ya terminó
                // → CLOSURE
  });
});
// Cada botón "Eliminar" elimina exactamente su li gracias al closure
```

10 · ERRORES TÍPICOS DE EXAMEN — TRAMPAS

1. Olvidar .trim() al validar

Un campo con solo espacios ' ' pasa if (valor === '') pero no debería.

```
if (valor === '') // MAL: ' ' pasa
if (valor.trim() === '') // BIEN: ' ' no pasa
```

2. Olvidar return tras el error

Sin return, muestra el error pero sigue ejecutando y añade el elemento vacío.

```
if (valor === '') {
  errorMsg.textContent = 'Error.';
  // falta return – SIGUE y rompe todo
}
if (valor === '') {
  errorMsg.textContent = 'Error.';
  return; // BIEN: corta aquí
}
```

3. var en lugar de const/let

El profesor penaliza var en la rúbrica. Usa const para referencias DOM, let para contadores.

```
var nombre = 'Ana'; // MAL – penaliza calidad
const nombre = 'Ana'; // BIEN – no se reasigna
let count = 0; // BIEN – se reasigna con ++
```

4. && vs || en tramos numéricos

Para que un número esté en un rango necesita cumplir LAS DOS condiciones a la vez → &&. Con || siempre es true.

```
if (nota >= 5 || nota <= 6.9) // MAL: siempre true
if (nota >= 5 && nota <= 6.9) // BIEN: entre 5 y 6.9
```

5. Leer .value fuera del evento

El valor se captura al cargar la página, cuando el input está vacío.

```
const val = input.value; // MAL: siempre ''
btn.addEventListener('click', function() {
  const val = input.value; // BIEN: valor actual
});
```

6. NodeList no tiene .sort()

querySelectorAll devuelve NodeList. Para ordenar, convertir a Array primero.

```
document.querySelectorAll('li').sort(...) // TypeError
[...document.querySelectorAll('li')].sort(...) // BIEN
```

7. Olvidar e.preventDefault() en submit

Sin él, la página se recarga y pierdes todo el estado.

```
form.addEventListener('submit', function(e) {
  e.preventDefault(); // PRIMERA línea – siempre
});
```

8. Botón antes de li en appendChild

Si añades el li al DOM antes de terminar de montarlo, puede quedar incompleto.

```
// MAL
lista.appendChild(li);
li.appendChild(deleteBtn); // demasiado tarde en algunos casos

// BIEN – montar todo antes de añadir al DOM
li.appendChild(span);
li.appendChild(deleteBtn);
lista.appendChild(li);      // al final
```

11 · PATRONES COMPLETOS DE LOS EJERCICIOS

Patrón: Añadir a lista con total (Carrito / Alumnos)

```
// 1. Leer y validar
const nombre = input.value.trim();
const precio = parseFloat(priceInput.value);
if (nombre === '') { errorMsg.textContent = '...'; return; }
if (nombre.length < 3) { errorMsg.textContent = '...'; return; }
if (isNaN(precio)||precio<=0){ errorMsg.textContent = '...'; return; }
errorMsg.textContent = '';

// 2. Crear nodo
const li = document.createElement('li');
li.dataset.price = precio;

const span = document.createElement('span');
span.textContent = nombre + ' - ' + precio.toFixed(2) + '€';
li.appendChild(span);

const btn = document.createElement('button');
btn.textContent = 'Eliminar';
btn.addEventListener('click', function() { li.remove(); actualizarTotal(); });
li.appendChild(btn);
lista.appendChild(li);

// 3. Limpiar y actualizar
input.value = ''; priceInput.value = ''; input.focus();
actualizarTotal();
```

Patrón: Búsqueda en tiempo real (Buscador / Agenda)

```
searchInput.addEventListener('input', function() {
  const query = searchInput.value.toLowerCase();
  document.querySelectorAll('#lista li').forEach(function(li) {
    const texto = li.dataset.nombre.toLowerCase();
    li.style.display = texto.includes(query) ? 'list-item' : 'none';
  });
});
```

Patrón: Generador aleatorio con lista (Quiniela)

```
lista.innerHTML = ''; // limpiar lista anterior
const opciones = ['1', 'X', '2'];
let unos = 0, equis = 0, doses = 0;

for (let i = 1; i <= 14; i++) {
  const res = opciones[Math.floor(Math.random() * 3)];
  if (res === '1') unos++;
  if (res === 'X') equis++;
  if (res === '2') doses++;

  const li = document.createElement('li');
  li.textContent = 'Partido ' + i + ': ' + res;
  lista.appendChild(li);
}

count1.textContent = unos;
countX.textContent = equis;
```

```
count2.textContent = doses;
totalGeneradas++;
```

Patrón: Función reutilizable con parámetro (Propinas)

```
function calcular(porcentaje) {
  const cuenta = parseFloat(importe.value);
  if (importe.value === '' || isNaN(cuenta) || cuenta <= 0) {
    errorMsg.textContent = 'Importe inválido.'; return;
  }
  errorMsg.textContent = '';
  propina.textContent = (cuenta * porcentaje / 100).toFixed(2) + '€';
  total.textContent = (cuenta * (1 + porcentaje/100)).toFixed(2) + '€';
}
btn10.addEventListener('click', function() { calcular(10); });
btn15.addEventListener('click', function() { calcular(15); });
btn20.addEventListener('click', function() { calcular(20); });
```

12 · CHULETA RÁPIDA — TODO LO ESENCIAL

Seleccionar por ID	<code>document.getElementById('id')</code>
Seleccionar primero	<code>document.querySelector('.clase')</code>
Seleccionar todos	<code>document.querySelectorAll('li')</code>
Crear elemento	<code>document.createElement('li')</code>
Añadir texto	<code>element.textContent = 'texto'</code>
Añadir hijo	<code>parent.appendChild(child)</code>
Eliminar elemento	<code>element.remove()</code>
Leer input	<code>input.value / input.value.trim()</code>
Limpiar input	<code>input.value = ''</code>
Ocultar	<code>el.style.display = 'none'</code>
Mostrar li	<code>el.style.display = 'list-item'</code>
Añadir clase	<code>el.classList.add('clase')</code>
Quitar clase	<code>el.classList.remove('clase')</code>
¿Tiene clase?	<code>el.classList.contains('clase')</code>
Guardar en dataset	<code>li.dataset.price = valor</code>
Leer dataset	<code>parseFloat(li.dataset.price)</code>
Deshabilitar botón	<code>btn.disabled = true</code>
Vaciar lista	<code>ul.innerHTML = ''</code>
Aleatorio 1-100	<code>Math.floor(Math.random()*100)+1</code>
Aleatorio 0-2	<code>Math.floor(Math.random()*3)</code>
2 decimales	<code>(valor).toFixed(2)</code>
String a número	<code>parseFloat(str) / parseInt(str)</code>
¿Es número?	<code>!isNaN(valor)</code>
Quitar espacios	<code>str.trim()</code>
A minúsculas	<code>str.toLowerCase()</code>
¿Contiene?	<code>str.includes('texto')</code>
Primeros N chars	<code>str.substring(0, N)</code>
NodeList → Array	<code>[...document.querySelectorAll('li')]</code>
Prevenir recarga	<code>e.preventDefault()</code>

